

BARD

BEHAVIOURALLY ADAPTIVE REACTIVE DIALOGUE

BRADLEY CHARLES · Supervisor: TAYLOR PAPKE · Academic Capstone 2026

A Godot 4 RPG in which a locally-running LLM rewrites every NPC's dialogue each in-game night based on what the player did that day — no cloud, no API keys, no internet required.

Godot 4.6

GDScript

Python 3 Pipeline

Ollama · Gemma 4 E4B

Local GPU Inference

No Cloud Dependencies

PROJECT OVERVIEW

BARD investigates whether a locally-running LLM can create the *illusion of memory* in NPC dialogue without cloud infrastructure. The player hunts monsters, completes bounties, and sleeps at the inn. Every night, a Python pipeline reads the day's events and prompts Ollama to write fresh, context-aware dialogue for each named NPC before morning.

The academic focus is the **LLM pipeline** — not the gameplay. The bounty system, combat, and economy exist to feed the model meaningful, structured context.

KEY FEATURES

- **Nightly dialogue regeneration** — every NPC speaks fresh every morning
- **NPC memory** — recollections extracted each night, injected into future prompts
- **Rumor propagation** — Chronicle pipeline derives town rumors from kill history
- **Local inference only** — Ollama + Gemma 4 E4B, no API keys required
- **Robust fallback chain** — retry → LLM repair → previous day → static fallback
- **Full bounty loop** — 36 contracts across 4 monster families feed the LLM context
- **Dynamic world gen** — `world_gen.py` bootstraps lore, NPC names & personalities

SYSTEM ARCHITECTURE



THE LLM PIPELINE

Every night a **Python subprocess** reads the day's game state and prompts a locally-running LLM to write fresh dialogue for each named NPC.

```
Player sleeps at Inn
→ end_day() writes game_state.json
→ Godot spawns end_of_day.py subprocess
→ Pipeline reads kills, bounties, flags, NPC memory
→ Ollama (Gemma 4 E4B) generates dialogue JSON
→ Validates schema → LLM repair if malformed → fallback
→ Writes dialogue/{npc_id}_day{N}.json
→ LLM extracts one NPC recollection → appends to state
→ Writes pipeline_ready.flag
→ Godot detects flag → reloads dialogue → go to town
```

NPC MEMORY & CHRONICLE

Memory: After generating each NPC's dialogue, a second LLM call extracts one subjective recollection and appends it to `game_state.json`. Future prompts inject these facts, building narrative persistence across days.

"Aerin returned with slime cores — seven of them. She said the pack was larger than expected."
— Example recollection stored after Day 3

Chronicle (Ctrl+R): Reads full kill history, generates a weekly narrative, then derives rumors (75% named, 25% anonymous) injected into all future NPC prompts — giving NPCs awareness of events they didn't witness.

DIALOGUE ARCHITECTURE

LLM content is **merged** with hardcoded role menus at runtime — the LLM controls narrative; the engine controls system reliability.

Hardcoded: Sleep · Shop · Bounty Board · Turn In · Goodbye
LLM-generated: greeting node + all branching conversation trees

Validation enforces `greeting` + `farewell` nodes, no broken `next` refs. Failure triggers an LLM repair call, then falls back to the previous day's file, then to `pipeline/fallbacks/`.

NPC variants per role 3 personalities (A/B/C)
Named NPCs Innkeeper · Blacksmith · Guild Commander

THE GAME IN ACTION



LLM-generated NPC dialogue in Thornwall



Bounty Board — 36 contracts across 4 zones

TECHNOLOGY STACK

Game Engine	Godot 4.6 · GDScript
LLM Runtime	Ollama — local GPU
LLM Model	Gemma 4 E4B
Pipeline	Python 3 subprocess
IPC	JSON + sentinel flag files
UI Architecture	100% GDScript · signal-driven
Resolution	1920×1080 · Forward+ renderer

SOURCE CODE



GitHub Repository
github.com/BradleyCharles/BARD
Full source including Godot project, Python pipeline, NPC archetypes, and dialogue fallbacks.